HOA SEN UNIVERSITY
RESPECT DIVERSITY

Mảng một chiều (One-Dimensional Array)

Nhập môn lập trình

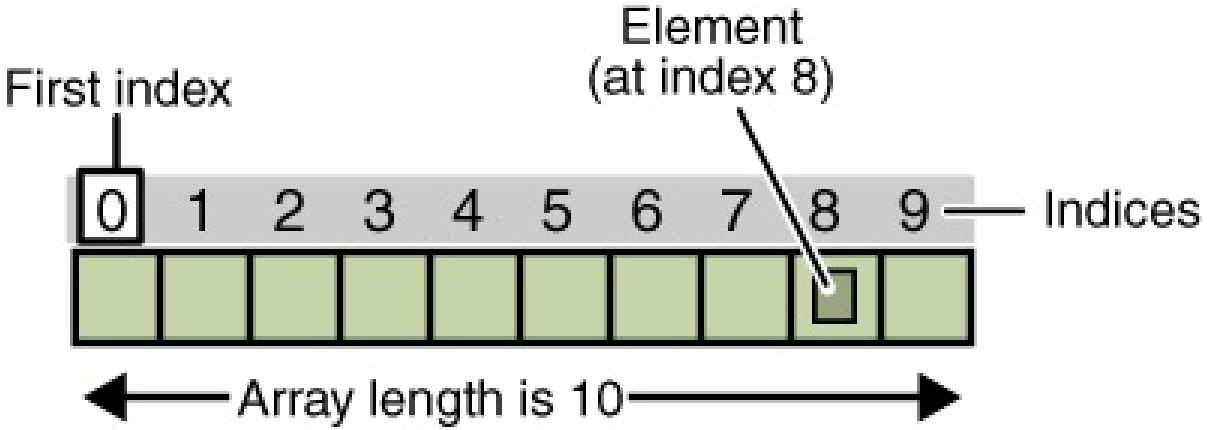
Vũ Đình Khôi

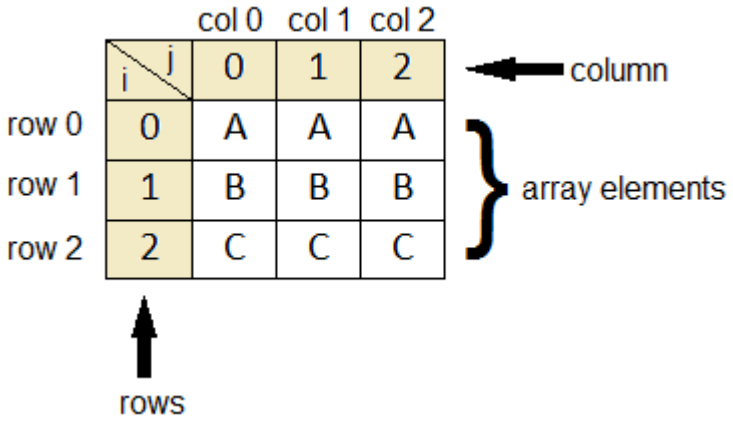
Nội dung

- Khái niệm mảng
- Mảng một chiều
  - Khai báo
  - Khởi tạo mảng
  - Duyệt mảng
  - Nhập mảng/Xuất mảng
  - Sắp xếp mảng bằng thuật toán BubbleSort
  - Các ví dụ về mảng 1 chiều

Khái niệm mảng

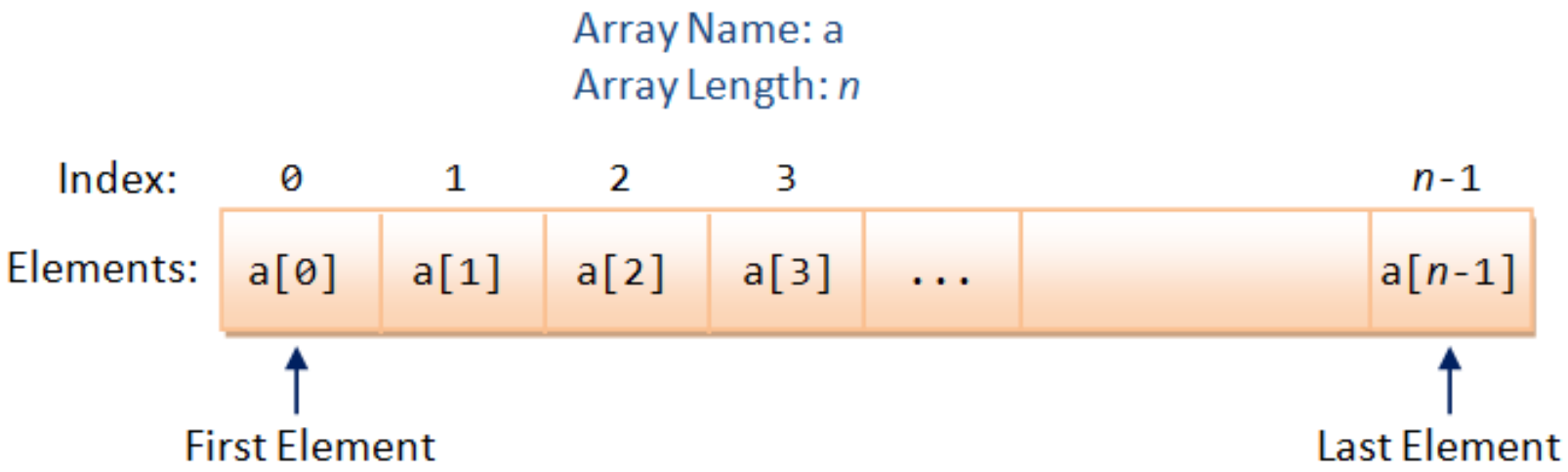
- Mảng là một tập hợp nhiều phần tử có cùng kiểu và chung một tên
  - Các phần tử được lưu trữ trên các ô nhớ liên tiếp nhau
  - → do đó, phải xác định được chiều dài (số phần tử) trong mảng.
- Phần tử mảng (element)
- Chỉ số mảng (index/indices)
- Mảng 1 chiều (one-dimensional)
- Mảng nhiều chiều (multi-dim)

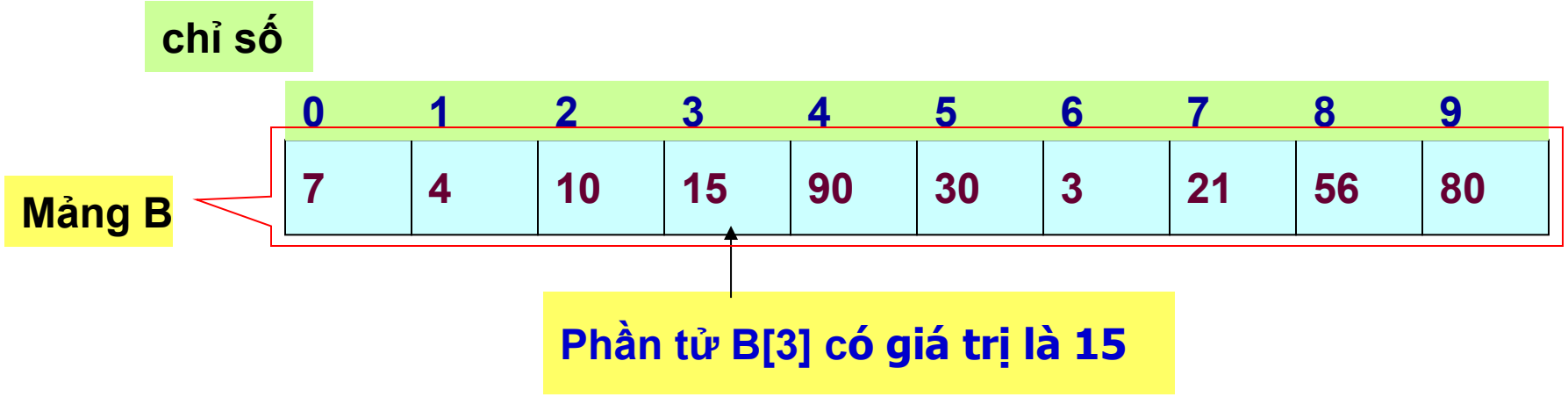




Mảng 1 chiều (1-dimensional array)

- Mảng một chiều là mảng mà những phần tử của nó được xác định bởi một chỉ số.





Mảng 1 chiều (1-dimensional array)

- Khai báo (declaration)

```
type arrayName[arraylength]
```

- Khai báo tường minh (phải chỉ định số phần tử trong mảng)

```
int marks[5]; // mảng số nguyên lưu điểm gồm 5 phần tử
double numbers[20];
#define SIZE 10 // hay const int SIZE = 10;
float temps[SIZE];
```

Mảng 1 chiều (1-dimensional array)

- Khởi tạo (initialization) = khai báo + gán giá trị cho mảng

  type arrayName[arraylength] = {...}

```c
/* Cau truc mang */
#include <stdio.h>

int main() {
    int arr[10] = {5, 10, 15, 20, 25, 30, 35, 40, 45, 50};
    int i;
    for (i = 0; i < 10; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }
    return 0;
}
```

D:\Codes\C\array_01.exe

```
arr[0] = 5
arr[1] = 10
arr[2] = 15
arr[3] = 20
arr[4] = 25
arr[5] = 30
arr[6] = 35
arr[7] = 40
arr[8] = 45
arr[9] = 50
```

Mảng 1 chiều (1-dimensional array)

- Khởi tạo (initialization) tất cả các phần tử trong mảng là zero (0)

  type arrayName[arraylength] = {0}

```c
/* Cau truc mang */
#include <stdio.h>

#define SIZE 10

int main() {
    int arr[SIZE] = {};  // tat ca cac phan tu zero (0)
    int i;
    for (i = 0; i < SIZE; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }
    return 0;
}
```

Định nghĩa số phần tử trong mảng

D:\Codes\C\array_02.exe

```
arr[0] = 0
arr[1] = 0
arr[2] = 0
arr[3] = 0
arr[4] = 0
arr[5] = 0
arr[6] = 0
arr[7] = 0
arr[8] = 0
arr[9] = 0
```

Mảng 1 chiều (1-dimensional array)

- Duyệt mảng (traverse) và xử lý từng phần tử trong mảng
  - Duyệt xuôi: từ đầu mảng đến cuối mảng

```c
/* Cau truc mang */
#include <stdio.h>

#define SIZE 10

int main() {
    int arr[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40, 45, 50};
    int i;
    // Duyet tung phan tu trong mang tu dau den cuoi (xuoi)
    for (i = 0; i < SIZE; i++) {
        // Xu ly cac phan tu trong mang
        if (arr[i] % 3 == 0)  // kiem tra chia het cho 3
            printf("arr[%d] = %d\n", i, arr[i]);
    }
    return 0;
}
```

```
D:\Codes\C\array_03.exe
arr[2] = 15
arr[5] = 30
arr[8] = 45
```

Mảng 1 chiều (1-dimensional array)

- Duyệt mảng (traverse) và xử lý từng phần tử trong mảng
  - Duyệt ngược: từ cuối mảng lên đầu mảng

```c
/* Cau truc mang */
#include <stdio.h>

#define SIZE 10

int main() {
    int arr[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40, 45, 50};
    int i;
    // Duyet tung phan tu trong mang tu cuoi len dau (nguoc)
    for (i = SIZE - 1; i >= 0; i--) {
        // Xu ly cac phan tu trong mang
        if (arr[i] % 3 == 0)  // kiem tra chia het cho 3
            printf("arr[%d] = %d\n", i, arr[i]);
    }
    return 0;
}
```

```
D:\Codes\C\array_03a.exe
arr[8] = 45
arr[5] = 30
arr[2] = 15
```

Mảng 1 chiều (1-dimensional array)

- Nhập vào mảng gồm 5 số nguyên, và in ra giá trị các phần tử.

```c
/* Cau truc mang */
#include <stdio.h>
#define SIZE 5

int main() {
    int arr[SIZE];
    int i;
    // Nhap cac phan tu vao mang
    for (i = 0; i < SIZE; i++) {
        printf("Enter arr[%d] = ", i);
        scanf("%d", &arr[i]);
    }
    // Xuat gia tri cac phan tu
    for (i = 0; i < SIZE; i++) {
        printf("arr[%d] = %d; ", i, arr[i]);
    }

    return 0;
}
```

D:\Codes\C\array_04.exe

```
Enter arr[0] = 3
Enter arr[1] = 5
Enter arr[2] = 8
Enter arr[3] = 1
Enter arr[4] = 2
arr[0] = 3; arr[1] = 5; arr[2] = 8; arr[3] = 1; arr[4] = 2;
```

Mảng 1 chiều (1-dimensional array)

- Thuật toán Bubble sort để sắp xếp các phần tử trong mảng
  - So sánh từ cặp phần tử liền kề. Hoán vị nếu 2 phần tử không đứng đúng vị trí
  - Sau mỗi lượt (round) → phần tử lớn nhất/hoặc nhỏ nhất về cuối mảng (đúng vị trí)
  - Lặp lại các bước trên cho các phần tử còn lại trong mảng (trừ phần tử đã đúng vị trí)

Mảng 1 chiều: Bubble sort

```c
/* Sap xep mang (Bubble sort) */
#include <stdio.h>
#define SIZE 5

int main() {
    int a[SIZE] = {6, 8, 5, 1, 3};
    int i, j, temp;
    // Sap xep
    for (i = 0; i < SIZE - 1; i++) {
        for (j = 0; j < SIZE - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                // Hoan vi (swap)
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
    // In ket qua
    for (i = 0; i < SIZE; i++)
        printf("arr[%d] = %d; ", i, a[i]);
    return 0;
}
```

```
= 5; arr[3] = 6; arr[4] = 8;
```

Bài tập mảng 1 chiều

- Bài tập 1 (Giải tại lớp)
- Viết chương trình nhập n phần tử số nguyên vào mảng. Xuất ra màn hình
  - Tất cả các phần tử
  - Phần tử max trong mảng
  - Phần tử min trong mảng
  - Tổng các phần tử của mảng.
- Lưu ý
  - Viết bằng hàm

Bài tập mảng 1 chiều

```c
/* Cau truc mang */
#include <stdio.h>
#define SIZE 5

// Khai bao cac nguyen mau ham
void inputArray(int a[], int n);
void outputArray(int a[], int n);
int min(int a[], int n);
int max(int a[], int n);
int sum(int a[], int n);

// Dinh nghia cac ham
void inputArray(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        printf("a[%d]: ", i);
        scanf("%d", &a[i]);
    }
}
```

```c
void outputArray(int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d, ", a[i]);
    printf("\n");
}

int min(int a[], int n) {
    int min = a[0];
    int i;
    for (i = 1; i < n; i++)
        if (a[i] < min)
            min = a[i];
    return min;
}
```

Bài tập mảng 1 chiều

```c
int max(int a[], int n) {
    int max = a[0];
    int i;
    for (i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    return max;
}

int sum(int a[], int n) {
    int sum = 0;
    int i;
    for (i = 0; i < n; i++)
        sum += a[i];
    return sum;
}
```

```c
int main() {
    int n = SIZE;
    int a[n];
    inputArray(a, n);
    outputArray(a, n);
    printf("min: %d\n", min(a,n));
    printf("max: %d\n", max(a,n));
    printf("sum: %d\n", sum(a,n));

    return 0;
}
```

```
min: 2
max: 7
sum: 24
```

Bài tập mảng 1 chiều

- Bài tập 2: Viết chương trình nhập n phần tử số nguyên vào mảng. Thực hiện các yêu cầu sau:
  - Xuất mảng đó ra màn hình
  - Thêm 1 phần tử vào cuối mảng
  - Thêm 1 phần tử vào đầu mảng
  - Thêm 1 phần tử vào ví trí bất kỳ
  - Xuất lại mảng ra màn hình
- Sử dụng hàm, sử dụng menu để người dùng lựa chọn

Bài tập mảng 1 chiều

- Bài tập 3: Viết chương trình nhập n phần tử số nguyên vào mảng. Rồi thực hiện các yêu cầu sau:
  - Xuất mảng đó ra màn hình
  - Xoá phần tử đầu tiên của mảng
  - Xóa phần tử cuối cùng của mảng
  - Xóa phần tử ở vị trí bất kỳ của mảng
  - Xuất lại mảng ra màn hình

Bài tập mảng 1 chiều

1. Viết chương trình nhập 1 mảng số nguyên có n phần tử, rồi tìm phần tử nhỏ nhất, lớn nhất của mảng
2. Viết chương trình nhập 1 mảng số nguyên có n phần tử, tính tổng của các phần tử chẵn, tổng các phần tử lẻ.
3. Viết chương trình nhập 1 mảng số nguyên có n phần, tính tổng của các phần tử dương, tổng các phần tử âm.
4. Viết chương trình nhập một mảng số thực có n phần tử, hãy tính giá trị trung bình của các phần tử trong mảng đó, làm tròn 2 số lẻ.

Bài tập mảng 1 chiều

5. Viết chương trình nhập một mảng ký tự có n phần tử, hãy kiểm tra xem có bao nhiêu phần tử trùng nhau trong mảng.
6. Viết chương trình tạo một mảng số nguyên ngẫu nhiên có n phần tử, các phần tử này nằm trong khoảng từ -10 đến 10. Xuất mảng này ra màn hình theo thứ tự giảm dần.
7. Viết chương trình tạo một mảng số thực ngẫu nhiên có n phần tử, các phần tử này nằm trong khoảng từ 1.0 đến 2.0. Xuất mảng này ra màn hình theo thứ tự tăng dần.